CN Practical 12: Write a C Program for Parity Check Error Detection

Aim: To write a C program to implement the parity-check method for error detection in data communication.

Theory:

Parity checking is a simple error-detection technique in which an additional bit, known as the parity bit, is attached to a group of data bits. The parity bit helps the receiver determine whether the received data may have been corrupted during transmission. There are two common types of parity: even parity and odd parity.

In even parity, the total number of 1s in the data and parity bit must be even. In odd parity, the total number of 1s must be odd. The sender calculates the parity bit before transmitting the data, and the receiver checks the parity of the received bits. If the parity rule is violated, an error is detected. However, parity checking may fail to detect errors when an even number of bits change.

Algorithm:

1. Start the program.
2. Declare an array to store eight data bits.
3. Read eight binary bits from the user.
4. Count the number of 1s.
5. Calculate the even-parity bit.
6. Display the entered data and parity bit.
7. Stop the program.

Program in C:

```
#include <stdio.h>

int main()
{
    int data[8], i, count = 0, parity;

    printf("Enter 8 data bits: ");

    for (i = 0; i < 8; i++)
    {
        scanf("%d", &data[i]);

        if (data[i] == 1)
            count++;
    }

    parity = count % 2;

    printf("Data: ");

    for (i = 0; i < 8; i++)
        printf("%d", data[i]);

    printf("\nEven parity bit: %d\n", parity);

    return 0;
}
```

Sample output:

```
Enter 8 data bits: 1 0 1 1 0 0 1 0
Data: 10110010
Even parity bit: 0
```

Procedure:

1. Open a C compiler or IDE.
2. Type the program and save it with a .c extension.
3. Compile the program and correct any errors.
4. Execute the program.
5. Enter eight binary data bits, separated by spaces.
6. Count the number of 1s in the data.
7. Calculate the even-parity bit and display it.
8. Verify that the total number of 1s, including the parity bit, is even.

Observation:

The program accepts eight binary data bits and counts the number of 1s. It calculates the parity bit and displays it with the original data. The result follows the even-parity rule, although a receiver-side verification program would be needed to test received data for errors.

Conclusion:

A C program for calculating the parity bit was implemented successfully. The working of even parity and its role in detecting certain transmission errors was studied. The practical demonstrated a basic error-detection technique used in data communication.